

High-Level Design (HLD)

Project Title: RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop

1. Executive Overview

This document presents the high-level design of a customer support assistant powered by Retrieval-Augmented Generation (RAG). The system processes internal PDF knowledge bases, converts them into embeddings, retrieves semantically relevant information, and generates accurate responses using a Large Language Model (LLM).

Retrieval-Augmented Generation (RAG) enhances LLM performance by retrieving relevant document chunks at query time and injecting them into the prompt. This ensures responses are grounded in real data and significantly reduces hallucinations.

When the system detects low confidence or sensitive queries, it routes the request to a human agent through a Human-in-the-Loop (HITL) mechanism.

Business Goals

- Reduce repetitive customer support workload
 - Improve response speed and consistency
 - Ensure answers are grounded in trusted documents
 - Enable seamless human escalation
 - Build a scalable AI support system
-

2. Problem Statement

Organizations store critical knowledge in PDFs such as manuals, FAQs, onboarding guides, and SOPs. However, retrieving information from these documents is inefficient and time-consuming.

Key Challenges

- Slow manual search
 - Repetitive support queries
 - Inconsistent responses from agents
 - Traditional chatbots lack contextual understanding
 - No intelligent escalation system
-

3. Scope of Version 1

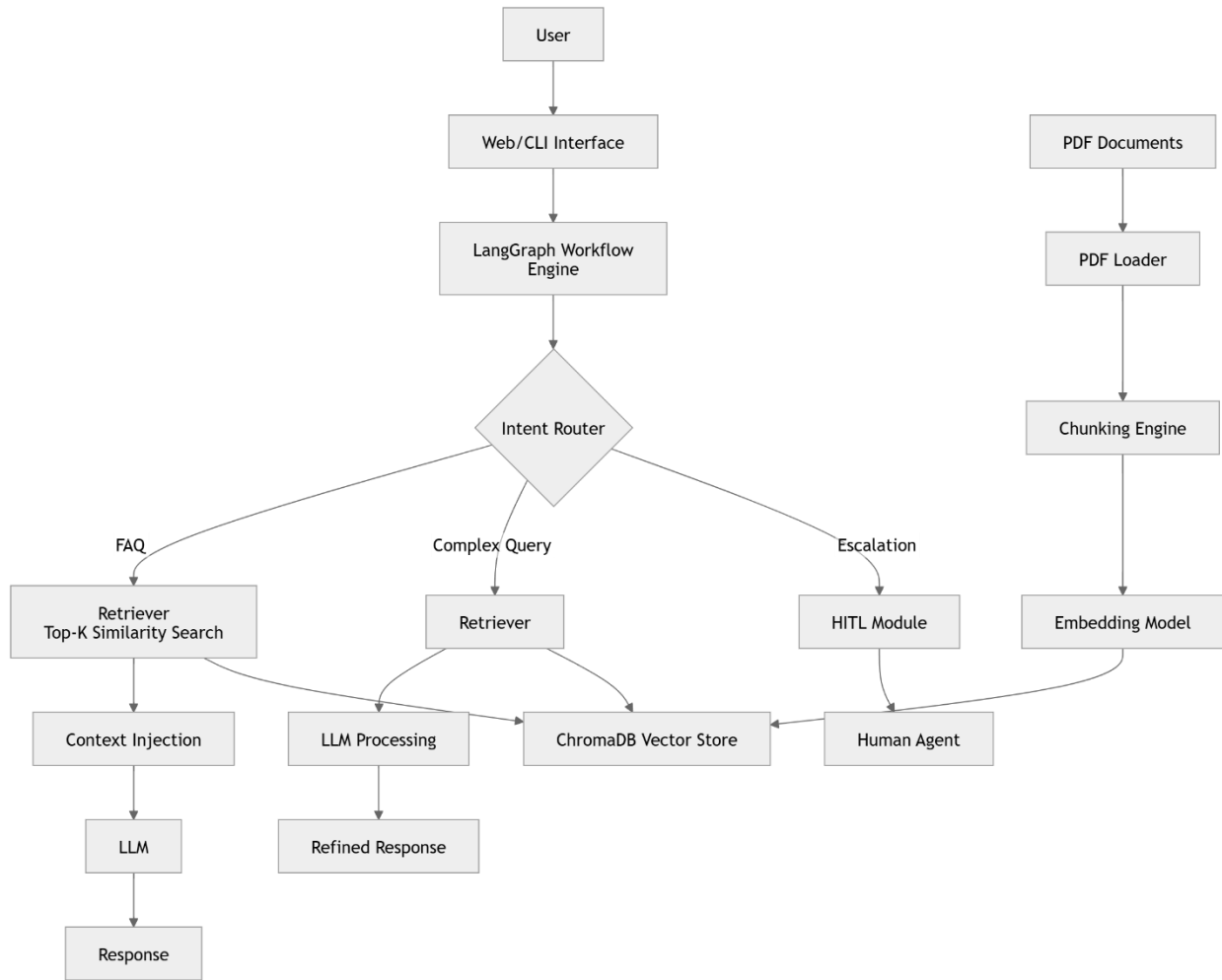
In Scope

- PDF ingestion and processing
- Text extraction and chunking
- Embedding generation
- Vector storage using ChromaDB
- Semantic retrieval system
- LLM-based response generation
- Workflow orchestration using LangGraph
- Human-in-the-Loop escalation
- Basic Web or CLI interface

Out of Scope

- Voice-based interaction
 - CRM integrations
 - Mobile applications
 - External web search
 - Multilingual support
-

4. Architecture Diagram



5. Core Components

- **User Interface:** Accepts user queries via web or CLI
- **PDF Loader:** Extracts text from uploaded documents
- **Chunking Engine:** Splits text into overlapping segments for better context retention
- **Embedding Layer:** Converts text into vector representations
- **Vector Store (ChromaDB):** Stores embeddings for efficient similarity search
- **Retrieval Service:** Performs Top-K similarity search using cosine similarity
- **LLM Layer:** Generates responses using retrieved context (context injection)
- **LangGraph Workflow Engine:** Models the system as a directed graph with conditional routing
- **Routing Layer:** Classifies query intent (FAQ, complex, escalation)
- **HITL Module:** Escalates complex or low-confidence queries to human agents

6. End-to-End Data Flow

Ingestion Flow

PDF → Text Extraction → Chunking → Embedding → ChromaDB

Query Flow

User Query → Intent Detection → Retriever (Top-K similarity search) → Context Injection → LLM → Response

If confidence is low or intent is escalation → Routed to HITL

7. Technology Choices

- **Backend:** Python
 - **RAG Framework:** LangChain
 - **Workflow Engine:** LangGraph
 - **Vector Database:** ChromaDB
 - **LLM:** OpenAI GPT or open-source models
 - **UI:** Flask or Streamlit
-

8. Scalability Strategy

Handling Large Documents

- Batch embedding generation
- Incremental indexing
- Optimized chunk size

Handling High Query Load

- API load balancing
- Caching
- Async processing

Reducing Latency

- Limit Top-K retrieval
 - Optimize prompt size
-

9. Security Considerations

- Secure API keys
 - Validate uploads
 - Restrict admin access
 - Logging and monitoring
-

10. Future Enhancements

- Multi-language support
 - Feedback loop
 - CRM integration
 - Analytics dashboard
 - Voice support
-

11. Conclusion

This architecture provides a scalable and reliable approach to building a customer support assistant using RAG. It ensures accurate, context-aware responses while supporting human escalation when needed.